Lab 5: Static Code Analysis

Name: Bhoomika Dayanand Jituri
SRN: PES1UG25CS807

1. Objective

To improve the quality, security, and style of the library-loan Python program by using Pylint, Bandit, and Flake8; correcting identified issues; and verifying the changes with new analysis reports.

2. Static-analysis results

Tool	Before	After
Pylint	6.07/10	10.00/10
Bandit	2 findings: 1 low, 1 medium	No issues identified
Flake8	13 reported style/import/exception issues	No violations reported (empty report)

3. Issue documentation table

Issue	Type	Original line(s)	Description	Fix applied
Mutable default argument	Bug / maintainability	9	dates=[] reused the same list across calls that omitted the argument.	Changed the default to None and initialized a new list inside the function.
Bare except	Reliability / style	16–22; 21	A bare except could hide unrelated programming errors.	Caught only the expected KeyError and ValueError when a loan or book is missing.
Unsafe eval()	Security	44–45; 45	eval() can execute arbitrary Python expressions supplied as a rule.	Replaced eval() with an AST-based evaluator that accepts only numeric constants and approved arithmetic operators.
Missing module/function docstrings	Code quality	1; 9, 16, 24, 27, 33, 39, 44, 47, 52	The code did not explain the module and function responsibilities.	Added a module docstring and function docstrings.
Function naming	Style	27	renewBook did not follow the recommended snake_case convention.	Renamed it to renew_book and updated the call site.

Global fine-rate mutation	Maintainability	28	renewBook used global to modify FINE_PER_DAY.	Moved the rate into a mutable FINE_CONFIG dictionary and updated its per_day value.
File handles and encoding	Reliability / portability	34, 40	Files were opened without a context manager or explicit encoding.	Used with open(..., encoding='utf-8') for reading and writing JSON.
Unused import	Style	2	The os import was not used.	Removed the unused import.
Old-style string formatting and long lines	Style	14, 31	Percent formatting/concatenation and long lines reduced readability and violated style checks.	Used f-strings and wrapped long expressions across lines.
Negative overdue fine	Functional correctness	Original calculate_fine function	Negative overdue days produced a negative fine.	Clamped overdue days to zero before calculating the fine.
Non-string book ID in sample data	Data consistency	Main demonstration	The demonstration passed integer 4521 even though book IDs are treated as strings.	Converted book IDs to strings when recording loans.

4. Reflection

What is static code analysis, and why is it useful?

Static code analysis examines source code without executing it. It can reveal likely bugs, unsafe patterns, maintainability problems, and style violations early, making code easier to review and maintain.

What did Pylint identify in this program?

Pylint identified issues including a mutable default argument, missing docstrings, a bare except, a non-snake_case function name, global-state mutation, file handling without explicit encoding/context managers, an unused import, and older string formatting. The score improved from 6.07/10 to 10.00/10 after the fixes.

What did Bandit identify, and how were the findings addressed?

Bandit identified a try/except/pass pattern (B110) and use of eval() (B307). The exception handling was narrowed to expected exceptions, and eval() was replaced with a restricted AST-based arithmetic evaluator. The final Bandit report identified no issues.

What did Flake8 contribute?

Flake8 reported an unused import, missing blank lines, long lines, and a bare except. These style and formatting issues were corrected. The final Flake8 report was empty.

What was the most important security lesson?

Avoid evaluating untrusted strings as code. eval() can run arbitrary Python expressions. When a program only needs arithmetic rules, it is safer to parse the expression and explicitly allow only the required syntax and operators.

What limitations should be kept in mind?

A clean static-analysis report does not prove that software is free of bugs. Tests and code review are still important. The arithmetic evaluator intentionally supports only a limited set of operators, and application behaviour should be tested when its rules or data formats change.

How did verification demonstrate that the fixes worked?

The corrected program compiled and ran. Pylint reported 10.00/10, Bandit reported no issues, and Flake8 reported no violations. Before-and-after reports were saved for comparison.

5. Git commit evidence

Commit	Message
c95df1e	Add baseline static analysis reports
70d95be	Fix static analysis issues in library system
9d75505	Add final static analysis verification reports